

Centrum Projektowe

Oficjalna dokumentacja projektu, stan wdrożenia i kierunek rozwoju

Wersja dokumentu: 0.3

Data: 8 października 2026

Status: obowiązujący dokument kierunkowy po Etapie 6 i analizie architektury „Po Kapiemu”

Repozytorium: kapi1944/Centrum_Projektowe

Charakter aplikacji: prywatna, local-first, osobista pamięć projektowa i centrum dowodzenia

Spis treści

Spis treści

Centrum Projektowe

Oficjalna dokumentacja projektu, stan wdrożenia i kierunek rozwoju

1. Streszczenie wykonawcze
2. Źródła i zakres analizy

CZĘŚĆ I — STAN CENTRUM PROJEKTOWEGO

3. Aktualny stan funkcjonalny

CZĘŚĆ II — CO „PO KAPIEMU” JUŻ ROZWIĄZAŁO

4. Mapa Po Kapiemu jako źródło wzorców
5. Integracja z Ogarniaczem — element do przejścia niemal wprost

CZĘŚĆ III — CO PRZENOSIMY, CO TYLKO INSPIRUJE, CO ODRZUCAMY

6. Macierz decyzji

CZĘŚĆ IV — DOCELOWA ARCHITEKTURA CENTRUM

7. Zasada local-first pozostaje
8. StorageProvider zamiast Google Drive
9. Raspberry Pi jako prywatny hub
10. Model pliku i dokumentu

CZĘŚĆ V — UX, DESIGN I JAKOŚĆ

11. Design System Centrum 2.0 — adaptacja, nie klon
12. Wyszukiwanie — wzorzec do szybkiego przeniesienia
13. CI i regresje

CZĘŚĆ VI — INTEGRACJE

14. Centrum ↔ Ogarniacz
15. Centrum ↔ Po Kapiemu
16. GitHub
17. Sztuczna inteligencja

CZĘŚĆ VII — ZAKTUALIZOWANY PLAN ETAPOWY

18. Nowa oficjalna roadmapa

19. Co robić natychmiast, a czego nie robić

CZĘŚĆ VIII — ZASADY ARCHITEKTONICZNE OBOWIĄZUJĄCE OD TEJ REWIZJI

20. Dziesięć zasad

CZĘŚĆ IX — RYZYKA I PUNKTY KONTROLNE

21. Rejestr głównych ryzyk

CZĘŚĆ X — PYTANIA DECYZYJNE

Pytanie 1

Pytanie 2

Pytanie 3

Pytanie 4

Pytanie 5

Pytanie 6

Pytanie 7

Pytanie 8

Pytanie 9

Pytanie 10

Pytanie 11

Pytanie 12

Pytanie 13

Pytanie 14

Pytanie 15

22. Źródła techniczne

23. Status dokumentu

1. Streszczenie wykonawcze

Centrum Projektowe przestało być koncepcją „kolejnego menedżera zadań”. W obecnym kierunku jest to **prywatny system pamięci projektowej**, którego najważniejszą funkcją jest przechwycenie nieuporządkowanej informacji, zachowanie źródła, przekształcenie jej w propozycje do weryfikacji oraz utrzymywanie aktualnego modelu projektu bez utraty historii.

Rdzeń z Etapów 0-5 jest już funkcjonalny: projekty, Skrzynka, punkt powrotu „Gdzie skończyłem?”, analiza wpisu, rejestr decyzji, zastępowanie ustaleń, analiza wpływu, obszary, etapy, siedem rodzajów pracy, pytania i blokady. Etap 6 został wykonany lokalnie jako commit `5c6acd8 feat: add backup and restore: 11 magazynów IndexedDB` ma pełny eksport/import, walidację, wykrywanie konfliktów i atomowe odtwarzanie. Według raportu wykonawczego przechodzi 97/97 testów. Commit nie został jeszcze wypchnięty na `origin/main`.

Analiza „Po Kapiemu” prowadzi do dwóch równoległych wniosków:

1. **Warto przejąć dopracowane wzorce techniczne i UX**, zwłaszcza Design System 2.0, dostępny mobilny shell, szybkie wyszukiwanie Ctrl+K, małe komponenty UI, wzorce bezpieczeństwa backendu, CI oraz kontrakt integracji z Ogarniaczem.
2. **Nie wolno przejąć jego historycznych ograniczeń architektury danych**: biznesowe dane rozproszone między statycznymi plikami, Reactem i `localStorage` nie pasują do Centrum. Centrum ma już lepszy fundament: IndexedDB, transakcje, migracje i rozdzielenie domeny od adapterów.

Docelowy kierunek to nadal **local-first**, ale z przygotowaną warstwą `StorageProvider / SyncProvider`. Raspberry Pi pracujące 24/7 z dodatkowym SSD ma zostać prywatnym hubem: magazynem plików, miejscem kopii, lekkiego API i późniejszej synchronizacji. Dostęp zdalny powinien domyślnie odbywać się przez Tailscale, bez wystawiania prywatnego Centrum publicznie do Internetu.

Najważniejsza zasada dalszego rozwoju brzmi:

Po Kapiemu jest biblioteką sprawdzonych wzorców. Nie jest szablonem, który należy skopiować w całości.

2. Źródła i zakres analizy

Dokument opiera się na:

- aktualnym repozytorium `kapi1944/Centrum_Projektowe` na `origin/main` — commit `53d6c28` ;
- lokalnym raporcie wykonania Etapu 6 — commit `5c6acd8` , bez push;
- repozytorium `kapi1944/Po_Kapiemu` , ze szczególnym uwzględnieniem gałęzi `codex/etap-00-5-stabilizacja` , której HEAD to `5d84723` ;
- dostarczonej mapie „Po Kapiemu — mapa architektury i przepływów”, stan z 8 października 2026;
- plikach `docs/audyt/ETAP_00_5_STABILIZACJA.md` i `docs/design/DESIGN_SYSTEM_2.md` z „Po Kapiemu”;
- kodzie Layout, wyszukiwania, autoryzacji, statusów projektu, rejestru sekcji, backendu Express, Docker Compose i pipeline CI;
- oficjalnej dokumentacji Tailscale, Seafile i Raspberry Pi dotyczącej Linux/Raspberry Pi, ARM64 i pamięci SSD.

2.1. Ważne rozróżnienie stanów repozytoriów

`Centrum_Projektowe` na GitHubie nadal wskazuje `53d6c28` , natomiast Etap 6 został wykonany lokalnie jako `5c6acd8` i nie został wypchnięty. Dokument traktuje Etap 6 jako **wykonany lokalnie, oczekujący na push**.

`Po_Kapiemu` ma stary `main` , ale aktualna mapa i audyt odnoszą się do gałęzi `codex/etap-00-5-stabilizacja` . Dlatego właśnie ta gałąź jest głównym punktem odniesienia dla wzorców, a nie `main` .

CZĘŚĆ I — STAN CENTRUM PROJEKTOWEGO

3. Aktualny stan funkcjonalny

3.1. Etapy ukończone

Etap	Zakres	Stan
0	Fundament React + TypeScript + Vite, routing, kontrakt repozytorium	✓ ukończony
1	Pamięć projektu, historia, atomowy zapis	✓ ukończony
2	Skrzynka, szybki wpis, „Gdzie skończyłem?”	✓ ukończony
3R	Analiza wpisu, weryfikacja, provenance	✓ ukończony
4	Decyzje, zastępowanie, analiza wpływu	✓ ukończony
4.5	Polonizacja i uporządkowanie interfejsu	✓ ukończony
5	Obszary, etapy, elementy pracy, pytania, blokady	✓ ukończony
6	Backup/restore 11 magazynów, konflikty, atomowy import	✓ ukończony lokalnie, bez push

3.2. Obecny przepływ wiedzy

```
Surowy wpis
↓
Niezmienny oryginał
↓
AnalizaWpisu
↓
Weryfikacja użytkownika
↓
Zastosowanie tylko zaakceptowanych elementów
├─> propozycja decyzji
├─> analiza wpływu
├─> element pracy
├─> otwarte pytanie
↓
Historia i dalsza realizacja
```

Kluczowa reguła domenowa pozostaje obowiązująca:

ORYGINAŁ ≠ ANALIZA ≠ DECYZJA. ANALIZA NIE JEST PRAWDĄ.

3.3. Mocne strony obecnej architektury

- domena jest oddzielona od Reacta i IndexedDB;
- źródła i pochodzenie informacji są przechowywane jawnie;
- operacje krytyczne korzystają z transakcji;
- migracje IndexedDB zachowują wcześniejsze dane;
- decyzja jest odrębną encją od zadania;
- zmiana decyzji nie usuwa historii;
- analiza wpływu wymaga jawnego zatwierdzenia;
- realizacja rozróżnia siedem rodzajów pracy zamiast sprowadzać wszystko do TODO;
- backup z Etapu 6 jest atomowy i wykrywa konflikty;
- cały interfejs użytkownika ma pozostać po polsku.

3.4. Najważniejsze aktualne ograniczenia

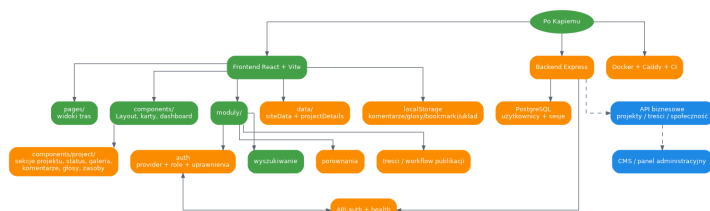
- dane są nadal związane z jedną przeglądarką i originem;
- brak współdzielonego magazynu plików;
- brak serwera Centrum i synchronizacji urządzeń;
- brak właściwego modelu Dokument/Plik/Repozytorium/Zasób;
- brak globalnego wyszukiwania po całej domenie;

- brak Karty zdrowia projektu;
 - brak systemu „Wymaga uwagi”;
 - brak prawdziwego LLM — działa dostawca regułowy;
 - repozytorium GitHub Centrum jest obecnie publiczne, mimo prywatnego przeznaczenia aplikacji;
 - realny browser smoke pobrania pliku backupu z Etapu 6 nie został jeszcze zweryfikowany.
-

CZĘŚĆ II — CO „PO KAPIEMU” JUŻ ROZWIĄZAŁO

4. Mapa Po Kapiemu jako źródło wzorców

Dostarczona mapa pokazuje dojrzały podział warstw: React/Vite na froncie, osobne moduły, część danych lokalnych, backend Express, PostgreSQL, Docker/Caddy, auth i planowane API biznesowe. Jej własny raport uczciwie wskazuje również największy problem: rozjazd źródeł danych między frontendem a backendem.



Mapa struktury technicznej Po Kapiemu

Wniosek dla Centrum: warto pożyczyć konstrukcję warstw i dopracowanie techniczne, ale nie model rozproszonej persystencji.

4.1. Dopracowany shell i nawigacja

`Layout.tsx` z gałęzi stabilizacyjnej ma kilka elementów szczególnie wartościowych:

- stały sidebar na desktopie;
- osobny nagłówek mobilny;
- `inert` dla nieaktywnego tła i zamkniętego menu;
- przywracanie focusu po zamknięciu;
- Escape i obsługa Tab/Shift+Tab;
- wspólny modal szybkiego wyszukiwania desktop/mobile;
- motyw systemowy/jasny/ciemny;
- stan compact/full;
- jawne stany sesji i błędów.

To są wzorce, które można **adaptować prawie bez zmiany koncepcji**, ale nawigacja musi być całkowicie inna, bo Centrum ma inną domenę.

4.2. Design System 2.0

„Po Kapiemu” ma ułożone tokeny semantyczne dla:

- tła i powierzchni;
- tekstu;
- obramowań;
- akcentu;
- sukcesu/ostrzeżenia/błędu/informacji;
- focusu;
- spacingu;
- radiusu;
- cieni;
- light/dark;
- responsywności.

Ma też małe, używane komponenty: `Przycisk`, `StatusBadge`, `PustyStan`.

Co przejąć: strukturę tokenów, zasady focus, spacing, dostępność i komponenty bazowe.

Czego nie kopiować: identycznej palety, marketingowej identyfikacji wizualnej i rozbudowanego CSS projektu publicznego.

4.3. Szybkie wyszukiwanie

Wyszukiwanie w „Po Kapiemu” to jeden z najlepszych kandydatów do adaptacji:

- Ctrl+K;
- natywny `dialog`;
- focus trap;
- Escape;
- przywracanie focusu;
- strzałki góra/dół + Enter;
- grupowanie wyników według typu;
- osobny widok wszystkich wyników.

Silnik jest prosty, deterministyczny i działa lokalnie. Centrum może zastosować ten sam wzorzec, ale indeksować inne typy: projekty, wpisy, decyzje, pytania, blokady, elementy pracy, dokumenty, zasoby i repozytoria.

4.4. Uprawnienia jako capability map

`uprawnienia.ts` mapuje role na możliwości zamiast rozrzucać warunki po komponentach.

To bardzo dobry wzorzec na przyszłość, ale pełna macierz ról „Po Kapiemu” nie ma sensu w prywatnym, jednoosobowym Centrum na obecnym etapie.

Rekomendacja: przejąć wzorzec `rola → capabilities`, ale początkowo mieć tylko:

- właściciel;
- ewentualnie przyszły gość/edytor;
- nie implementować społecznościowych ról typu moderator/wspierający/redaktor bez realnej potrzeby.

4.5. Backend Express i bezpieczeństwo

Backend „Po Kapiemu” ma wartościowe rozwiązania, które powinny być wzorcem dla Raspberry Pi Hub:

- `helmet`;
- limit JSON body;
- `httpOnly` cookie;
- `secure` w produkcji;
- `sameSite: strict`;
- hash tokenu sesji w bazie;
- rate limiting logowania;
- origin check dla operacji modyfikujących;
- `/api/health`;
- ogólne błędy 500 bez stack trace;
- osobne 400/413/404;
- poprawne zachowanie deep-linków SPA.

Nie należy jednak kopiować całego backendu ani auth 1:1. Hub Centrum ma inne zadania i prawdopodobnie zacząć jako prywatna usługa dostępna wyłącznie przez Ta-ilscale.

4.6. Docker + Caddy + PostgreSQL

`docker-compose.yml` w „Po Kapiemu” jest sensownym wzorcem wdrożeniowym:

```
PostgreSQL
↓
Express / aplikacja
↓
Caddy / HTTPS
```

Dla Centrum warto zachować **sposób organizacji usług, healthchecki i restart policy**, ale nie ma potrzeby natychmiast uruchamiać PostgreSQL na Raspberry Pi. Dopóki IndexedDB jest głównym źródłem prawdy, hub może zacząć jako lekki magazyn plików i kopii.

4.7. Rejestr sekcji projektu

`rejestrSekcjiProjektu.tsx` pozwala składać stronę projektu z modułowych sekcji i warunków widoczności. To dobry wzorzec dla późniejszej Karty projektu Centrum.

Nie powinno się jednak kopiować zestawu sekcji: komentarze, głosowania, społeczność i „zbuduj sam” są domeną publicznego portalu.

Dla Centrum analogiczny rejestr mógłby później składać sekcje:

- Gdzie skończyłem?;
- Zdrowie projektu;
- Decyzje;
- Realizacja;
- Pytania i blokady;
- Dokumentacja;
- Repozytoria;
- Zasoby;
- Historia.

4.8. Status, dojrzałość i postęp

W „Po Kapiemu” istnieją osobne mechanizmy statusu, kamieni milowych i dojrzałości. To ważna inspiracja dla Centrum:

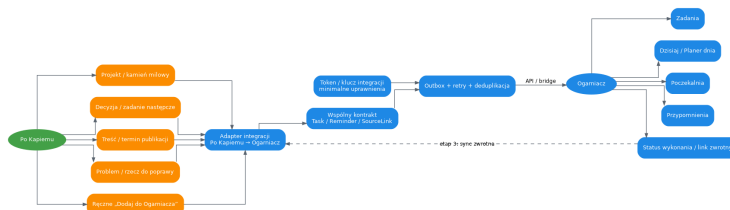
- **status projektu** odpowiada za tryb działania;
- **dojrzałość** mówi, na jakim poziomie jest koncepcja/produkt;

- **postęp** mierzy wykonanie;
- **zdrowie** ocenia kondycję organizacyjną.

Nie należy scalać tych pojęć w jeden procent.

5. Integracja z Ogarniaczem — element do przejęcia niemal wprost

Dostarczona mapa integracji proponuje dokładnie właściwy kierunek: najpierw jednokierunkowe tworzenie zadań/przypomnień, dopiero potem synchronizacja zwrotna.



Docelowa integracja Po Kapiem z Ogarniaczem

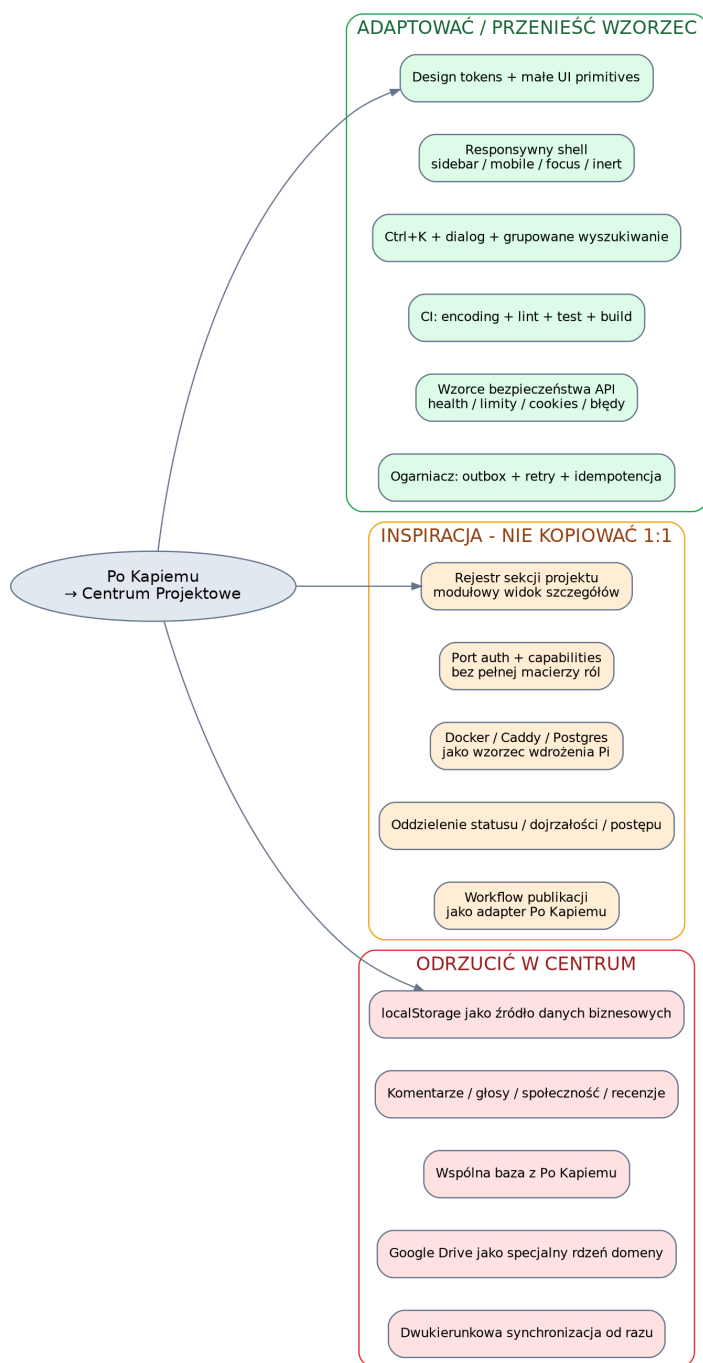
Najbardziej wartościowe elementy kontraktu:

- `sourceApp` ;
- `sourceType` ;
- `sourceId` ;
- `sourceUrl` ;
- tytuł/opis;
- `dueAt` / `remindAt` ;
- `priority` ;
- `target` ;
- `idempotencyKey` ;
- metadane i wersja kontraktu;
- outbox;
- retry;
- brak automatycznego wysyłania wszystkiego;
- minimalny token/klucz;
- synchronizacja zwrotna dopiero później.

Rekomendacja: Centrum powinno użyć tego samego standardu integracji. Różnić będą się tylko typy źródłowe, np. `element_pracy`, `blokada`, `decyzja`, `follow_up`, `pytanie`.

CZĘŚĆ III — CO PRZENOSIMY, CO TYLKO INSPIRUJE, CO ODRZUCAMY

6. Macierz decyzji



Ocena dziedziczenia rozwiązań Po Kapiemu

6.1. Adaptować bezpośrednio lub z niewielkimi zmianami

Element z Po Kapiemu	Przydatność	Zakres adaptacji
Design tokens i małe komponenty UI	bardzo wysoka	przenieść strukturę, zmienić identyfikację

Element z Po Kapiemu	Przydatność	Zakres adaptacji
Responsywny shell sidebar/mobile	bardzo wysoka	przenieść mechanikę focus/inert/Escape
Szybkie wyszukiwanie Ctrl+K	bardzo wysoka	wymienić źródła danych i typy wyników
GitHub Actions quality workflow	bardzo wysoka	dopasować skrypty Centrum
Wzorce backend security	wysoka	wykorzystać przy Raspberry Pi Hub
Health endpoint i healthcheck	wysoka	zastosować w hubie i kontenerach
Outbox + retry + idempotencja	bardzo wysoka	wspólny wzorzec integracji Ogarniacza

6.2. Traktować jako inspirację, nie kopię

Element	Dlaczego tylko inspiracja
<code>rejestrSekcjiProjektu</code>	Centrum ma inne sekcje i inne źródła danych
system uprawnień	dobry wzorzec capabilities, zbyt szeroka macierz ról
Express/PostgreSQL	może być przyszłym hubem, ale nie wolno porzucić local-first
status/dojrzałość/kamienie	dobra separacja pojęć, inne statusy domenowe
workflow publikacji	przyda się tylko do kontrolowanego eksportu do Po Kapiemu
Docker/Caddy	wartościowy wzorzec operacyjny, ale zależny od docelowego dostępu

6.3. Odrzucić

Element	Powód odrzucenia
localStorage jako trwałe dane biznesowe	Centrum ma już lepszą persistencję transakcyjną
społeczność, komentarze, głosy, recenzje	brak związku z prywatnym centrum dowodzenia
statyczne siteData jako źródło projektów	projekty Centrum są dynamiczne
jedna wspólna baza Po Kapiemu + Centrum	zbyt mocne sprzężenie prywatnego i publicznego systemu
automatyczna publikacja całego projektu	ryzyko ujawnienia prywatnych informacji
pełna dwukierunkowa synchronizacja od pierwszej wersji	bardzo wysoki koszt konfliktów i ryzyko błędów
Google Drive jako specjalny typ rdzenia	dostawca nie powinien kształtować domeny
kopiowanie całego wielkiego CSS / Layout 1:1	przeniosłoby także balast domeny publicznej

CZĘŚĆ IV — DOCELOWA ARCHITEKTURA CENTRUM

7. Zasada local-first pozostaje

Raspberry Pi nie powinno teraz zastąpić IndexedDB jako warunek działania aplikacji.

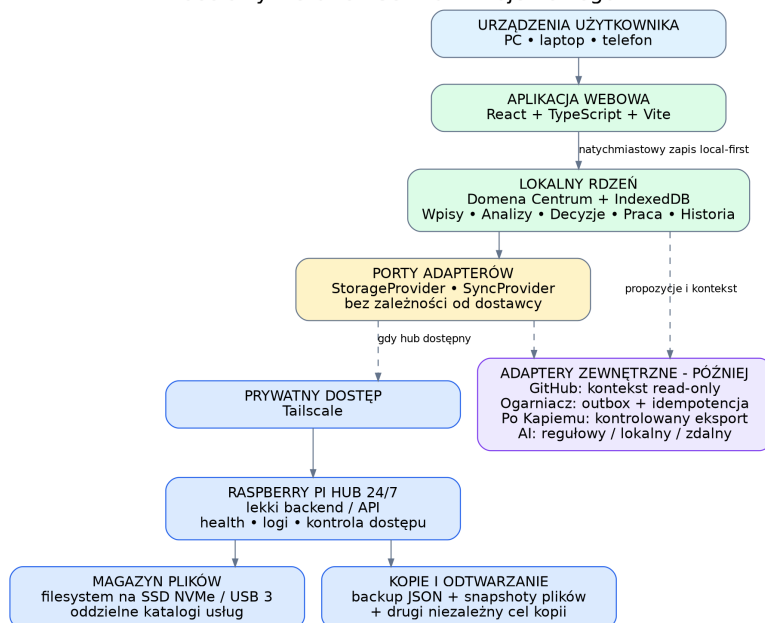
Zalecany model przejściowy:

```
przeglądarka
  ↓ natychmiast
IndexedDB
  ↓ opcjonalnie
StorageProvider / SyncProvider
  ↓ gdy dostępny
Raspberry Pi Hub
```

Dzięki temu:

- aplikacja uruchamia się bez serwera;
- wpis można zapisać natychmiast;
- awaria Pi nie blokuje pracy;
- można wdrażać hub etapami;
- nie trzeba teraz projektować pełnej synchronizacji multi-master.

Docelowy kierunek Centrum Projektowego



Docelowy kierunek architektury Centrum Projektowego

8. StorageProvider zamiast Google Drive

Domena powinna znać abstrakcję magazynu, nie konkretnego producenta.

Przykładowy kontrakt:

```

interface StorageProvider {
  zapiszPlik(...): Promise<PlikZdalny>
  pobierzPlik(...): Promise<Blob>
  usunPlik(...): Promise<void>
  pobierzMetadane(...): Promise<MetadanePliku>
  sprawdzDostepnosc(): Promise<StanProvidera>
}
  
```

Możliwe implementacje w przyszłości:

- `LocalMetadataProvider` — tylko metadane/odwołania;
- `RaspberryPiStorageProvider` — docelowy prywatny magazyn;
- `SeafileStorageProvider` — opcjonalny adapter;
- `WebDavStorageProvider` — opcjonalny adapter;
- dostawcy publicznej chmury — tylko jeśli kiedyś będą potrzebni.

Google Drive nie powinien mieć uprzywilejowanej pozycji w modelu domenowym.

9. Raspberry Pi jako prywatny hub

9.1. Rola

Rekomendowany wariant docelowy:

Raspberry Pi 24/7 = magazyn plików + kopie + lekkie API + przyszła synchronizacja.

Nie należy od razu budować pełnej „własnej chmury”. Pierwsza wersja hubu powinna być mała i przewidywalna.

9.2. Dysk SSD NVMe

Plan podłączenia SSD M.2 NVMe przez obudowę USB 3 jest zgodny z założeniem prywatnego magazynu. Warto jednak traktować dysk jako **główny magazyn**, a nie backup sam w sobie.

Proponowana struktura:

```
/storage/  
  centrum-projektowe/  
    pliki/  
    kopie/  
    manifesty/  
  ogarniacz/  
  osobisty-copilot/  
  wspolne-media/
```

Każda aplikacja powinna mieć własną przestrzeń logiczną i własny adapter.

9.3. Dostęp zdalny

Tailscale oficjalnie wspiera Raspberry Pi OS i systemy Debian-based. Dla prywatnego Centrum jest to lepszy punkt startowy niż publiczna domena i otwarte porty.

Pierwsza wersja:

PC / telefon / laptop



Tailscale



Raspberry Pi Hub

Publiczne HTTPS można rozważyć dopiero, jeżeli pojawi się realny przypadek użycia wymagający dostępu spoza tailnetu.

9.4. Seafile i Nextcloud

Seafile: technicznie atrakcyjny wariant opcjonalny; jego Docker wspiera ARM64. Może później dostarczyć gotowe wersjonowanie i klientów synchronizacji.

Nextcloud: funkcjonalnie bardzo szeroki, ale jako sam magazyn dla Centrum jest obecnie nadmiarowy. Dodaje więcej usług, paneli i obowiązków administracyjnych niż potrzebuje MVP.

Rekomendacja: zacząć od filesystem + lekkie API Centrum. Seafile zostawić jako plan B, jeśli szybko pojawi się potrzeba wygodnej synchronizacji plików niezależnie od aplikacji.

10. Model pliku i dokumentu

Plik fizyczny i dokument projektowy nie powinny być tym samym pojęciem.

10.1. Plik

Odpowiada za fizyczny obiekt:

- identyfikator;
- provider;
- ścieżkę/klucz;
- nazwę;
- MIME/type;
- rozmiar;
- checksum;
- wersję;
- datę modyfikacji.

10.2. Dokument

Odpowiada za znaczenie projektowe:

- projekt/obszar/etap;
- tytuł;
- opis;
- status aktualności;
- prywatność;
- powiązany Plik albo URL;
- powiązane decyzje;
- historia.

To pozwala np. temu samemu PDF-owi istnieć jako fizyczny plik na Pi, a w Centrum być „Specyfikacją konstrukcji — wymaga sprawdzenia”.

CZĘŚĆ V — UX, DESIGN I JAKOŚĆ

11. Design System Centrum 2.0 — adaptacja, nie klon

Najrozsądniejsze jest utworzenie w Centrum własnego małego systemu, bazującego na sprawdzonym modelu „Po Kapiemu”:

- tokeny semantyczne;
- focus 3 px;
- spacing 4/8/12/16/24/32/48;
- light/dark/system;
- małe komponenty Przycisk, PustyStan, StatusBadge, później KartaMetryki;
- responsywne karty i sidebar;
- reduced motion.

Nie kopiować nazwy marki, kategorii ani wizualnego charakteru publicznego portalu.

11.1. Proponowana nawigacja Centrum

```
Start / Dzisiaj
Skrzynka
Projekty
Wymaga uwagi
Decyzje
Realizacja
Dokumenty i pliki
Wyszukiwanie
Dane i kopie zapasowe
Ustawienia
```

Na małych ekranach można zastosować uproszczony header i drawer wzorowany mechanicznie na „Po Kapiemu”.

12. Wyszukiwanie — wzorzec do szybkiego przeniesienia

Docelowe globalne wyszukiwanie powinno działać z Ctrl+K i indeksować:

- projekty;
- wpisy;
- decyzje;
- obszary;
- etapy;
- elementy pracy;
- pytania;
- blokady;
- dokumenty;
- repozytoria;
- zasoby;
- historię — opcjonalnie.

Pierwsza wersja nie potrzebuje Elasticsearch ani serwera. Przy obecnej skali wystarczy lokalny indeks i deterministyczne filtrowanie.

13. CI i regresje

Pipeline „Po Kapiemu” jest bardzo dobrym kandydatem do przeniesienia prawie wprost.

Centrum powinno mieć GitHub Actions wykonujące co najmniej:

```
npm ci
sprawdzenie kodowania
lint
typecheck
testy
build
```

Po pojawieniu się backendu Pi:

```
check:server
smoke API
walidacja kontraktów backup/storage
```

Zasada: **nowe funkcje nie mogą zwiększać liczby testów dla samej liczby. Testujemy granice danych, migracje, transakcje i krytyczne przepływy.**

CZĘŚĆ VI — INTEGRACJE

14. Centrum ↔ Ogarniacz

Należy wykorzystać model z mapy „Po Kapiemu”, ale Centrum będzie naturalnie lepszym źródłem danych projektowych.

Przepływ MVP:

```
Element pracy / decyzja / follow-up / blokada
    ↓ ręczna akcja
„Wyślij do Ogarniacza”
    ↓
Outbox Centrum
    ↓ retry + idempotencyKey
Ogarniacz
    ↓
Zadanie / przypomnienie / poczekalnia / planer
```

Nie implementować od razu synchronizacji zwrotnej.

15. Centrum ↔ Po Kapiemu

Granica ma być odwrotna niż w klasycznej integracji danych:

Centrum jest prywatnym źródłem prawdy. Po Kapiemu dostaje wyłącznie wybrane, zatwierdzone materiały.

Przepływ:

```
Projekt prywatny
    ↓
Wybierz informacje do publikacji
    ↓
Kontrola klasyfikacji prywatności
    ↓
Podgląd: „Te informacje opuszczą Centrum”
    ↓
Eksport / API publikacyjne
    ↓
Po Kapiemu
```

Nigdy nie łączyć baz przez wspólne tabele i nie dawać Po Kapiemu dostępu do pełnego projektu.

16. GitHub

GitHub powinien być źródłem **kontekstu kodu**, a nie właścicielem projektu.

Warto pobierać:

- branche;
- ostatnie commity;
- Pull Requesty;
- stany CI;
- linki do plików.

Commit może być powiązany z decyzją lub elementem pracy, ale nie może automatycznie oznaczać decyzji jako „wdrożona”.

17. Sztuczna inteligencja

Obecny `RuleBasedAnalysisProvider` jest prawidłową warstwą przejściową.

Docelowo:

```
AnalysisProvider
├─ RuleBased
├─ LocalLLM
└─ RemoteLLM
```

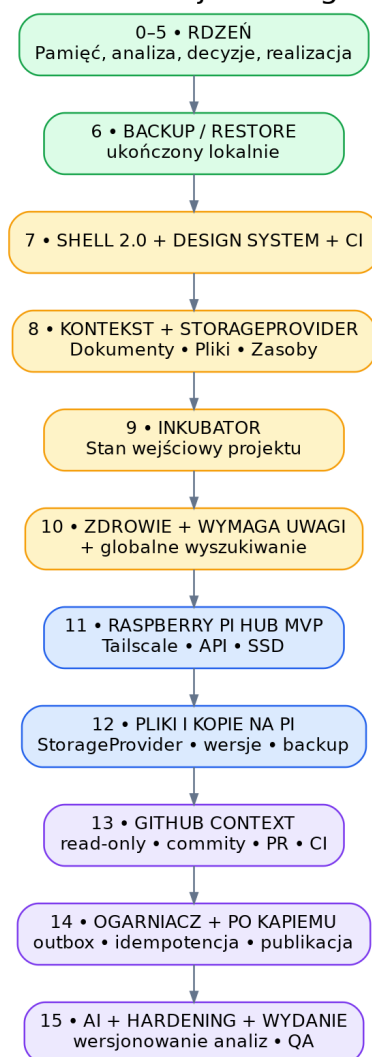
Każdy provider musi zwracać ten sam ustrukturyzowany model, a odpowiedź nadal przechodzi przez weryfikację użytkownika.

W przyszłości Raspberry Pi może hostować lekkie usługi pomocnicze, ale nie należy projektować całego sprzętu pod lokalne LLM-y.

CZĘŚĆ VII — ZAKTUALIZOWANY PLAN ETAPOWY

18. Nowa oficjalna roadmapa

Roadmapa Centrum Projektowego - Etapy 0-15



Roadmapa Etapów 0-15

Etapy 0-6 — fundament funkcjonalny

Stan: ukończone; Etap 6 lokalnie, bez push.

Rezultat:

- pamięć projektu;
- Skrzynka;

- analiza i weryfikacja;
 - decyzje i wpływ;
 - realizacja;
 - backup/restore.
-

Etap 7 — Shell 2.0, Design System i CI

Cel: wykorzystać dopracowane wzorce „Po Kapiemu” bez przenoszenia jego domeny.

Zakres:

- tokeny semantyczne Centrum;
- małe komponenty UI;
- dostępny sidebar/mobile drawer;
- light/dark/system;
- focus/inert/Escape;
- przygotowanie Ctrl+K;
- GitHub Actions quality workflow;
- brak przebudowy domeny.

Kryterium zakończenia: obecne funkcje działają identycznie, ale UI ma spójny system i kontrolę regresji CI.

Etap 8 — Kontekst projektu i `StorageProvider`

Zakres:

- `Dokument` ;
- `Plik` ;
- `RepozytoriumProjektu` ;
- `Zasób` ;
- `Kompetencja` ;
- relacje między projektami;
- klasyfikacja prywatności;
- kontrakty `StorageProvider` i `SyncProvider` ;
- provider lokalny/metadanych;
- backup nowych encji.

Nie implementować jeszcze Pi ani chmury.

Etap 9 — Inkubator / Stan wejściowy projektu

Centrum generuje i utrzymuje krótką strukturę:

- co chcemy osiągnąć;
- co wiemy;
- czego nie wiemy;
- założenia;
- decyzje do podjęcia;
- możliwe ścieżki;
- ryzyka;
- zależności;
- najbliższy sensowny krok.

Wszystko jako propozycje do weryfikacji.

Etap 10 — Zdrowie projektu, „Wymaga uwagi” i globalne wyszukiwanie

Zakres:

- Karta zdrowia;
 - zdrowie $\neq$ postęp;
 - kategorie applicable/not applicable;
 - ręczna korekta z powodem;
 - panel „Wymaga uwagi”;
 - decyzje do podjęcia;
 - globalne Ctrl+K na wzór Po Kapiemu;
 - grupowanie wyników.
-

Etap 11 — Raspberry Pi Hub MVP

Zakres:

- lekki Node/Express API;
- `/health`;
- prywatny dostęp przez Tailscale;
- katalog aplikacji na SSD;
- upload/download pliku;
- checksum;
- manifest;

- podstawowe logi;
 - wzorce bezpieczeństwa z Po Kapiemu;
 - bez pełnej synchronizacji bazy.
-

Etap 12 — Pliki i kopie na Raspberry Pi

Zakres:

- `RaspberryPiStorageProvider`;
- wysyłanie plików z Centrum;
- pobieranie;
- wersje plików;
- kopie backup JSON na Pi;
- drugi cel backupu;
- jawna obsługa braku hubu;
- brak blokowania pracy offline.

Opcjonalnie po tym etapie można zdecydować o Seafile.

Etap 13 — GitHub jako kontekst projektu

Zakres:

- read-only;
 - repozytoria, branche, commity, PR, CI;
 - ręczne powiązania z decyzją/pracą;
 - brak automatycznego „commit = wdrożone”.
-

Etap 14 — Ogarniacz i Po Kapiemu

Zakres:

- wspólny kontrakt `SourceLink`;
- outbox;
- retry;
- idempotencyKey;
- ręczne „Wyślij do Ogarniacza”;
- kontrolowany `PublicationSelection` do Po Kapiemu;
- podgląd informacji opuszczających Centrum;

- brak synchronizacji zwrotnej w pierwszej wersji.
-

Etap 15 — AI Providers, hardening i wydanie

Zakres:

- wersjonowane analizy;
 - LocalLLM/RemoteLLM za jednym kontraktem;
 - brak sekretów w frontendzie;
 - testy realnej przeglądarki;
 - backup/restore;
 - awarie hubu;
 - dostępność;
 - migracje;
 - monitoring;
 - dokumentacja bezpieczeństwa;
 - wydanie stabilnej wersji roboczej.
-

19. Co robić natychmiast, a czego nie robić

Teraz

1. Wypchnąć `5c6acd8` po upewnieniu się, że lokalny branch jest właściwy.
2. Ustawić repozytorium `Centrum_Projektowe` jako **Private**.
3. Wykonać realny browser smoke pobrania i importu backupu.
4. Rozpocząć Etap 7 — Design System/Shell/CI.
5. Dopiero potem modelować `Dokument / Plik / StorageProvider`.

Nie teraz

- pełny backend centralizujący całą bazę;
- multi-master sync;
- publiczna domena Centrum;
- Nextcloud jako zależność;
- Google Drive jako rdzeń;

- role społecznościowe;
 - komentarze/głosowania;
 - bezpośrednie wspólne tabele z Po Kapiemu;
 - automatyczna dwukierunkowa synchronizacja Ogarniacza;
 - kopiowanie całego kodu Po Kapiemu.
-

CZĘŚĆ VIII — ZASADY ARCHITEKTONICZNE OBOWIĄZUJĄCE OD TEJ REWIZJI

20. Dziesięć zasad

1. **Local-first jest domyślne.** Sieć rozszerza możliwości, nie warunkuje zapisu.
 2. **Oryginał jest niezmienny.** Analiza i decyzja są osobnymi warstwami.
 3. **AI proponuje, człowiek zatwierdza.**
 4. **Projekt nie jest listą zadań.** Decyzje, pytania, badania, blokady i dokumenty pozostają odrębne.
 5. **Provider zamiast vendora.** Domeny nie projektujemy pod Google Drive, Tailscale, Seafile czy GitHub.
 6. **Integracje są adapterami.** Nie łączymy baz aplikacji bezpośrednio.
 7. **Prywatne jest domyślne.** Publikacja wymaga jawnego wyboru.
 8. **Historia jest ważniejsza niż nadpisanie.** Zmiana kierunku zostawia ślad.
 9. **Krytyczne zapisy są atomowe.** Backup, import, decyzje i migracje nie mogą zostawiać połowy stanu.
 10. **Reuse selektywny.** Przenosimy sprawdzone małe wzorce z Po Kapiemu, nie cały portal.
-

CZĘŚĆ IX — RYZYKA I PUNKTY KONTROLNE

21. Rejestr głównych ryzyk

Ryzyko	Skutek	Ograniczenie
repo Centrum pozostaje publiczne	ujawnienie prywatnych informacji	zmiana na Private przed realnymi danymi
jeden SSD na Pi uznany za backup	utrata danych przy awarii nośnika	drugi niezależny cel kopii
zbyt wczesna synchronizacja	konflikty i utrata danych	najpierw backup/file storage, potem sync
skopiowanie dużej części Po Kapie-mu	sprzężenie i balast	małe, jawnie wybrane elementy
pełny Nextcloud/Seafile za wcześnie	czas administracji > wartość	filesystem + API jako MVP
automatyczne AI	błędne „fakty” w projekcie	obowiązkowa weryfikacja
backend jako jedyne źródło prawdy za wcześnie	utrata offline-first	IndexedDB pozostaje działającym rdzeniem
publiczny dostęp do Pi	zwiększona powierzchnia ataku	Tailscale-first

CZĘŚĆ X — PYTANIA DECYZYJNE

Poniższe pytania rozstrzygają decyzje, które nadal mają realny wpływ na Etapy 7–15. Sugerowana odpowiedź nie jest automatycznym ustaleniem — służy jako rekomendacja do zatwierdzenia.

Pytanie 1

Jaką docelową rolę ma pełnić Raspberry Pi?

- A. Tylko magazyn plików.
- B. Magazyn plików + automatyczne kopie Centrum.
- C. Magazyn plików + kopie + lekkie API + późniejsza synchronizacja urządzeń.

Sugerowana odpowiedź: C

Opcja	Zalety	Wady
B	+ prostsza administracja; szybki start; małe ryzyko architektoniczne	- nadal oddziela dane aplikacji od infrastruktury; później potrzebna kolejna przebudowa
C	+ tworzy prywatny hub całego ekosystemu; dobrze pasuje do Ogarniacza i Copilota	- wymaga API, monitoringu i później ostrożnie zaprojektowanej synchronizacji

Pytanie 2

Gdzie w najbliższych etapach ma pozostać główne źródło prawdy danych Centrum?

- A. IndexedDB; Pi na początku jest rozszerzeniem.
- B. Raspberry Pi od razu staje się centralnym serwerem danych.
- C. IndexedDB i Pi od razu są równorzędnymi źródłami multi-master.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ jeden centralny stan; prostsze odczyty z wielu urządzeń	- duży refaktor już teraz; aplikacja zaczyna zależeć od dostępności Pi
C	+ najlepsza praca offline na wielu urządzeniach	- najtrudniejszy wariant konfliktów, wersjonowania i odzyskiwania; duże ryzyko opóźnienia projektu

Pytanie 3

Jak ma wyglądać pierwszy fizyczny magazyn plików na Pi?

- A. Zwyczajny filesystem na SSD + lekkie API Centrum.
- B. Seafile jako podstawowy magazyn.
- C. Nextcloud/NextcloudPi jako podstawowy magazyn.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ gotowi klienci, wersjonowanie i synchronizacja; wspiera ARM64	- kolejna usługa, baza i API do utrzymania; Centrum staje się zależne od osobnej platformy
C	+ bardzo bogaty ekosystem osobistej chmury	- największy narzut administracyjny i funkcjonalny; wiele funkcji niepotrzebnych Centrum

Pytanie 4

Do czego ma służyć dodatkowy SSD pod Raspberry Pi?

- A. Wyłącznie Centrum Projektowe.
- B. Wspólny magazyn kilku prywatnych aplikacji z osobnymi katalogami.
- C. Ogólny NAS na wszystkie prywatne pliki.

Sugerowana odpowiedź: B

Opcja	Zalety	Wady
B	+ wykorzystuje sprzęt dla całego ekosystemu; zachowuje separację logiczną	- wymaga pilnowania uprawnień, limitów i strategii backupu wielu usług
C	+ maksymalne wykorzystanie pojemności i jeden domowy magazyn	- miesza krytyczne dane aplikacji z ogólnym NAS-em; większa powierzchnia awarii i trudniejsze odtwarzanie

Pytanie 5

Jak Centrum ma modelować pliki?

- A. Osobna encja Plik + Dokument + StorageProvider .
- B. Tylko zwykłe linki/ścieżki tekstowe.
- C. Przechowywanie binarnych plików bezpośrednio w IndexedDB/bazie.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ minimalny kod i szybka implementacja	- brak kontroli wersji, checksum, providera, przenoszenia i stanu aktualności
C	+ jedna baza zawiera wszystko	- słabe dla CAD, PDF, zdjęć i wideo; duże backupy i trudniejsze migracje

Pytanie 6

Jak uzyskiwać zdalny dostęp do Raspberry Pi?

- A. Tylko domowa sieć LAN.
- B. Tailscale na prywatnych urządzeniach.
- C. Publiczna domena HTTPS dostępna z Internetu.

Sugerowana odpowiedź: B

Opcja	Zalety	Wady
B	+ prywatna sieć, brak klasycznego port-forwardingu, proste dla PC/telefonu/laptopa	- każde urządzenie musi należeć do tailnetu
C	+ dostęp z dowolnej przeglądarki bez klienta VPN	- większa odpowiedzialność za auth, HTTPS, aktualizacje, monitoring i ataki z Internetu

Pytanie 7

Czy pliki z SSD mają być dostępne także poza interfejsem Centrum?

- A. Nie; tylko przez aplikację/API.
- B. Tak; dodatkowo jako zwykłe katalogi przez kontrolowany udział sieciowy.
- C. Tak; pełny odpowiednik dysku chmurowego z klientami desktop/mobile.

Sugerowana odpowiedź: B

Opcja	Zalety	Wady
B	+ CAD, PDF i grafiki można otwierać normalnymi programami; brak vendor lock-in	- zmiany poza Centrum trzeba wykrywać i uzgadniać z metadanymi
C	+ najwyższa wygoda użytkowa i synchronizacja plików	- praktycznie wymusza Seafile/Nextcloud i zwiększa zakres projektu

Pytanie 8

Jak zabezpieczyć dane SSD przed awarią?

- A. Jeden SSD bez dodatkowej kopii.
- B. Drugi niezależny nośnik/komputer z automatyczną kopią.
- C. Drugi nośnik + później zaszyfrowana kopia off-site.

Sugerowana odpowiedź: C docelowo; B jako minimum początkowe

Opcja	Zalety	Wady
B	+ tanio, prosto, bez limitów usług zewnętrznych	- oba nośniki w jednym miejscu nie chronią przed kradzieżą/pożarem
C	+ prawdziwa reguła 3-2-1 jest osiągalna; odporność na awarię lokalizacji	- wymaga szyfrowania, zarządzania kluczem i wyboru zewnętrznego celu

Pytanie 9

Jak podejść do uwierzytelniania w pierwszej wersji Raspberry Pi Hub?

- A. Tailscale + pojedynczy właściciel; proste tokeny aplikacyjne dopiero między usługami.
- B. Od razu skopiować pełne logowanie/sesje z Po Kapiemu.
- C. Od razu wdrożyć pełny RBAC z wieloma rolami.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ sprawdzony kod sesji, cookies i rate limiting już istnieje	- w jednoosobowym hubie to dodatkowa baza, UI i powierzchnia błędów przed realną potrzebą
C	+ gotowość do wielu użytkowników i ról	- najwyższy koszt i ryzyko nadprojektowania

Pytanie 10

Jak dużo Design Systemu Po Kapiemu przenieść do Centrum?

- A. Tokeny, komponenty bazowe i wzorce dostępności; osobna identyfikacja Centrum.
- B. Skopiować cały Design System 2.0 wraz z wyglądem.
- C. Zaprojektować wszystko od zera.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ najszybszy spójny wygląd i najmniej pracy projektowej	- Centrum wygląda jak moduł Po Kapiemu; przenosi niepotrzebne reguły i zależności wizualne
C	+ pełna swoboda i czysta tożsamość	- marnowanie już dopracowanej pracy i większe ryzyko ponownych błędów UX

Pytanie 11

Co zrobić z Layoutem i wyszukiwaniem Po Kapiemu?

- A. Zaadaptować mechanikę responsywną i Ctrl+K, ale zbudować nawigację Centrum od nowa.
- B. Skopiować Layout.tsx i stopniowo usuwać niepotrzebne elementy.
- C. Nie wykorzystywać żadnego kodu i zrobić inny model nawigacji.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ bardzo szybki widoczny rezultat	- łatwo przenieść zależności od auth, społeczności, topbara i klas CSS publicznego portalu
C	+ zero zależności historycznych	- tracimy przetestowane focus trap, inert, mobile menu i skróty klawiaturowe

Pytanie 12

Jak wykorzystać pipeline jakości Po Ka-piemu?

- A. Przenieść wzorzec GitHub Actions już w najbliższym etapie.
- B. Dodać CI dopiero przy pierwszym deployu na Pi.
- C. Zrezygnować z CI i polegać na lokalnych testach Codexa.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ mniej konfi-guracji teraz	- kolejne etapy mogą być pu-showane bez niezależnej kon-troli regresji
C	+ zero narzutu infrastruktural-nego	- brak niezależ-nego, powta-rzalnego po-twierdzenia ja-kości repozyto-rium

Pytanie 13

Jak wdrożyć integrację Centrum → Ogranicz?

- A. Ręczna akcja + wspólny kontrakt + outbox + retry + idempotencyKey.
- B. Bezpośredni request API po kliknięciu, bez outboxu.
- C. Pełna dwukierunkowa synchronizacja od pierwszej wersji.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ bardzo mało kodu i szybkie MVP	- awaria sieci może zgubić zamiar użytkownika; trudniejsza deduplikacja
C	+ pełna spójność statusów w obu aplikacjach	- konflikty, pętle synchronizacji i ukryte nadpisanie; zdecydowanie zbyt duży zakres na start

Pytanie 14

Jak Centrum ma przekazywać dane do Po Kapiemu?

- A. Jawny wybór publikowanych pól + podgląd + eksport/API.
- B. Wspólna baza projektów dla obu aplikacji.
- C. Automatyczna publikacja wybranych statusów całego projektu.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ brak duplikacji danych i natychmiastowa spójność	- łączy prywatne Centrum z publicznym portalem i zwiększa ryzyko wycieku
C	+ mało pracy użytkownika i szybkie aktualizacje publiczne	- trudniej kontrolować kontekst; ryzyko publikacji informacji nieprzeznaczonych do ujawnienia

Pytanie 15

Kiedy uruchomić prawdziwe modele AI?

- A. Dopiero po Storage/Health/Search; jako wymienny provider za istniejącą weryfikacją.
- B. Teraz, przed dalszą architekturą, aby AI zaczęło analizować każdy projekt.
- C. Zrezygnować z LLM i pozostać wyłącznie przy regułach.

Sugerowana odpowiedź: A

Opcja	Zalety	Wady
B	+ szybciej uzyskamy bogatsze klasyfikacje i podsumowania	- AI zacznie generować dane zanim model projektu i magazyn plików są ustabilizowane; ryzyko przepalenia czasu na integrację
C	+ pełna przewidywalność, prywatność i brak kosztów	- analiza naturalnych, chaotycznych wpisów pozostanie mocno ograniczona

22. Źródła techniczne

1. Repozytorium `kapi1944/Centrum_Projektowe` ,
`origin/main` — `53d6c28` (8.10.2026).
2. Lokalny raport wykonania Etapu 6 —
`5c6acd8 feat: add backup and restore` , 97/97 te-
stów, bez push.
3. Repozytorium `kapi1944/Po_Kapiemu` , gałąź `codex/`
`etap-00-5-stabilizacja` — HEAD `5d84723` .
4. `docs/audyt/ETAP_00_5_STABILIZACJA.md` — Po Ka-
piemu.
5. `docs/design/DESIGN_SYSTEM_2.md` — Po Kapiemu.
6. Dostarczony PDF „Po Kapiemu — mapa architektu-
ry i przepływów”, 8.10.2026.
7. Tailscale Docs — Install Tailscale on Linux: [https://](https://tailscale.com/docs/install/linux)
tailscale.com/docs/install/linux
8. Seafile Admin Manual — Supported architecture:
[https://manual.seafile.com/13.0/setup/architectu-](https://manual.seafile.com/13.0/setup/architecture/)
[re/](https://manual.seafile.com/13.0/setup/architecture/)
9. Raspberry Pi Documentation — SSDs: [https://](https://www.raspberrypi.com/documentation/accessories/ssds.html)
[www.raspberrypi.com/documentation/accessories/](https://www.raspberrypi.com/documentation/accessories/ssds.html)
[ssds.html](https://www.raspberrypi.com/documentation/accessories/ssds.html)

23. Status dokumentu

Ten plik zastępuje wcześniejsze robocze roadmapy Centrum Projektowego w zakresie kolejności Etapów 7-15 oraz strategii storage/infrastruktury.

Do czasu kolejnej rewizji obowiązuje jako główny kierunek:

local-first → selektywny reuse z Po Kapiemu → StorageProvider → Raspberry Pi Hub → bezpieczne integracje → AI.